

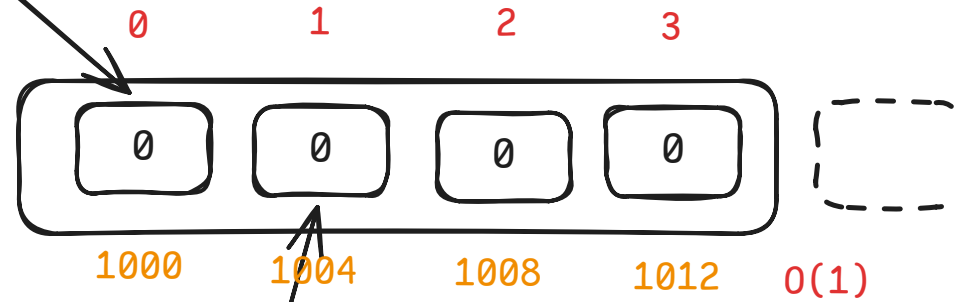
Lecture 02 - Arrays and slices

Arrays

the size of array is part of the type

```
var a [4]int32
```

32 bits = 4 bytes



$$a[1] = 1000 + 4 * i$$

```
var a int32  
var b int32  
var c int32  
var d int32
```

```
var a [3]int  
var b [4]int
```

```
a = b // compile error
```

```
len(a) // 3
```

Multidimensional array:

```
var twoDimensions [2][3]int
```

Slices

There are 3 common ways to create slices

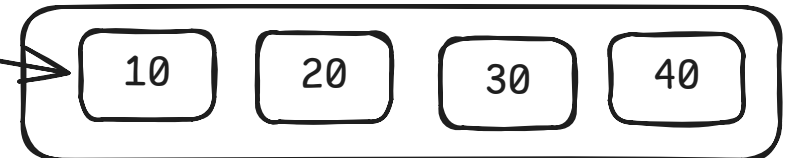
slice literal

1

```
numbers := []int{10, 20, 30, 40}
```

```
type SliceHeader struct {  
    Data *T  
    Len int  
    Cap int  
}
```

backing array



make

2

```
numbers := make([]int, 4)
```

[0 0 0 0]

```
numbers := make([]int, 4, 5)
```

capacity: how far slice may extend

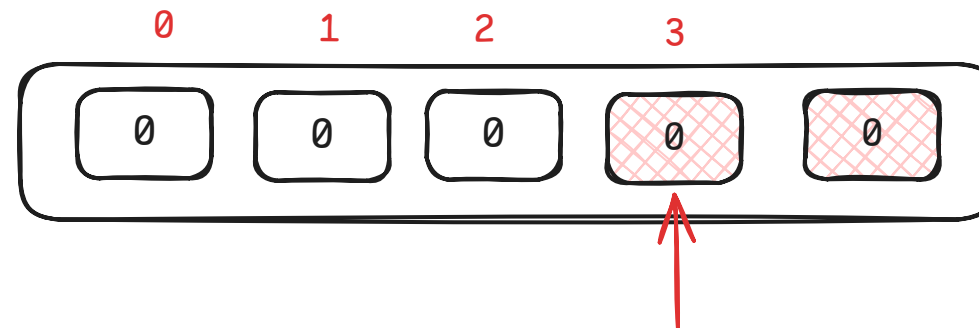
length: how many elements currently belong to the slice

```
numbers := make([]int, 3, 5)
```

```
len(s) // 3
```

```
cap(s) // 5
```

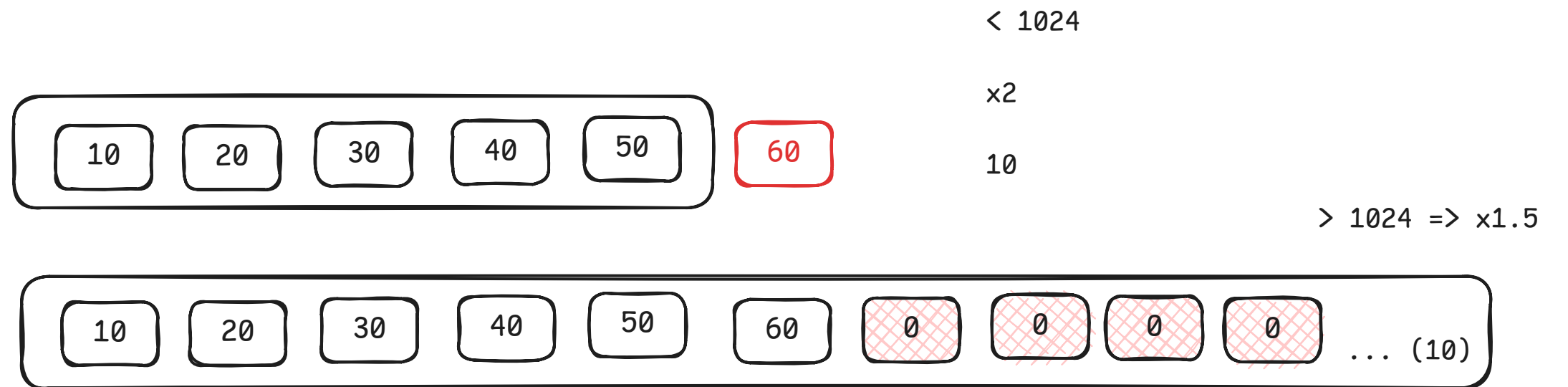
```
numbers[3] // panic
```



```
a := 60
```

```
numbers = append(numbers, a)
```

```
// [10 20 30 40 50 60]
```



From an array

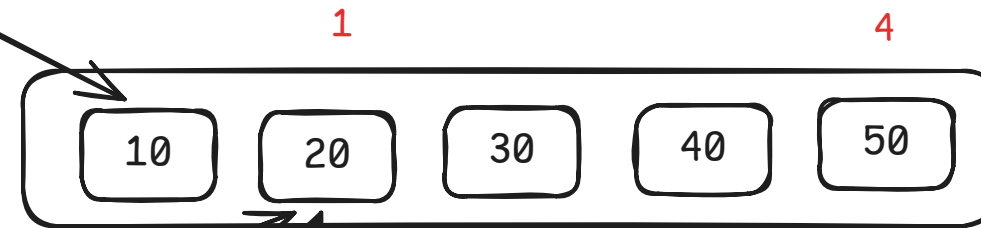
3

```
arr := [5]int{10, 20, 30, 40, 50}
```

```
numbers := arr[1:4]
```

```
type SliceHeader struct {  
    Data *T  
    Len int  
    Cap int  
}
```

```
numbers[0] -> 20  
numbers[1] -> 30  
numbers[2] -> 40
```



`s[start:end]`

start - inclusive
end - exclusive

`[1, 4)`

```
numbers := []int{10, 20, 30, 40, 50}
```

```
a := numbers[1:4]
```

```
b := numbers[2:5]
```

```
a[1] = 999
```

```
fmt.Println(b[0]) // 999
```

```
fmt.Println(a[1]) // 999
```

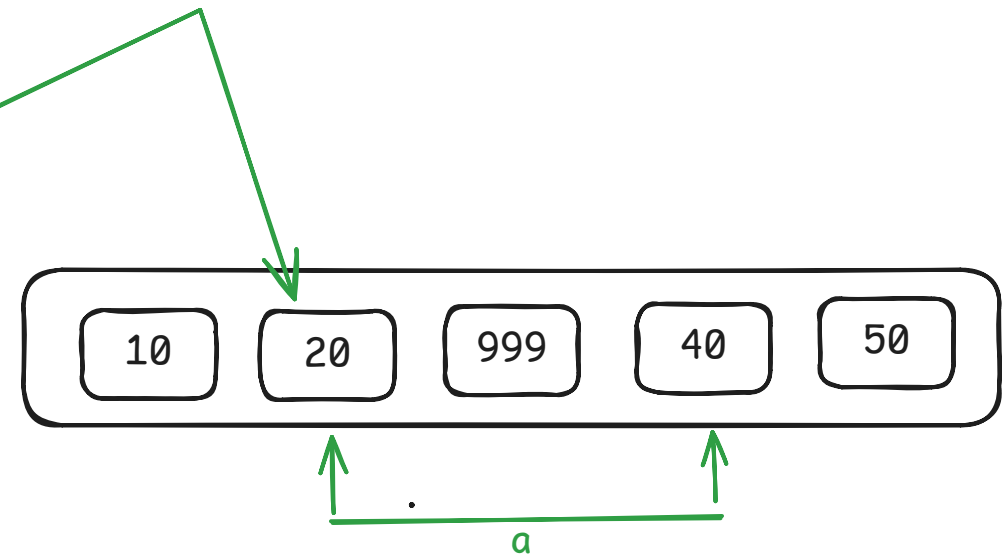
```
fmt.Println(numbers[2]) // 999
```

```
a = append(a, ...)
```

```
a[2] = 100
```

```
type User struct {  
    Name string  
}
```

```
var users []User
```



```
func change(s []int) {  
    s[0] = 100  
}  
  
func main() {  
    numbers := []int{10, 20, 30}  
    change(numbers)  
    fmt.Println(numbers) // [100 20 30]  
}
```

```
type SliceHeader struct {  
    Data *T  
    Len int  
    Cap int  
}
```

copy

```
type SliceHeader struct {  
    (copy) Data *T  
    Len int  
    Cap int  
}
```

